

## MILESTONE 3: API Logic Integration

In **Milestone 3**, the focus shifts to designing and integrating the backend API logic that powers document generation using the Gemini 1.5 Pro model. This milestone ensures a functional bridge between frontend inputs and the AI-powered document generation, which is the core of the LegalEase project.

---

### Activity 3.1: Create and Configure main.py

This step involves initializing the FastAPI application, setting up the base endpoint for health-checks, and including modular API routes.

#### Steps:

- Initialize the FastAPI app
- Define a root (/) endpoint to verify the service is running
- Include routes from the routes.py module

```
from fastapi import FastAPI
from legalEaseAPI.routes import router

app = FastAPI(title="LegalEase - AI Legal Document Generator")

# Include API routes
app.include_router(router)

# Root endpoint
@app.get("/")
def home():
    return {"message": "Welcome to LegalEase AI Legal Document Generator API"}

# Run FastAPI if executed directly
if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8000)
```

### Activity 3.2: Handle Document Generation Logic in routes.py

This task handles user inputs from the frontend and sends them to the Gemini model for document generation.

Steps:

- Define a Pydantic DocumentRequest model to validate incoming data.
- Initialize the GeminiDocumentGenerator from ai\_core.gemini\_generator.
- Create a POST endpoint (/generate) to accept user input and return generated legal content.

```
from fastapi import APIRouter
from pydantic import BaseModel
from ai_core.gemini_generator import GeminiDocumentGenerator

router = APIRouter()
gemini_generator = GeminiDocumentGenerator()

class DocumentRequest(BaseModel):
    document_type: str
    parties: str
    terms: str
    dates: str

@router.post("/generate")
def generate_legal_document(request: DocumentRequest):
    response = gemini_generator.generate_document(
        request.document_type,
        request.parties,
        request.terms,
        request.dates
    )
    return {"document": response}
```

## MILESTONE 4: Frontend Development – LegalEase

Milestone 4 focuses on creating a responsive and user-centric web interface for **LegalEase** using **Streamlit**. This milestone enables users to easily input legal details, generate AI-powered documents, preview the results, make inline edits, and download documents in multiple formats (TXT, DOCX, PDF) with rich formatting and embedded branding.

---

### Activity 4.1: Designing and Developing the User Interface

#### Step 1: Streamlit Page Configuration and Layout Setup

- Configured Streamlit app with:

```
st.set_page_config(page_title="LegalEase", layout="centered")
```

- Added a three-column layout using `st.columns` to center-align the company logo

```
col1, col2, col3 = st.columns([1, 2, 1])
with col2:
    st.image(WEB_LOGO_PATH, use_container_width=True)
```

#### Step 2: Header and Title

- Displayed a subtle center-aligned title under the logo:

```
st.markdown(
    "<h2 style='text-align: center;'>AI Legal Document Generator</h2>",
    unsafe_allow_html=True
)
```

#### Step 3: User Input Interface

- Created intuitive inputs for:
  - document\_type: type of legal document
  - parties: involved stakeholders
  - terms: semicolon-separated clauses
  - dates: effective date

```
document_type = st.text_input("Document Type")
parties = st.text_area("Parties Involved")
terms = st.text_area("Terms & Conditions (Use semicolons for bullet points)")
dates = st.text_input("Effective Date")
```

## Activity 4.2: Creating Dynamic Previews and Export Options

### Step 1: Generate Document with Backend AI

- On clicking "Generate Document", a POST request is sent to the FastAPI backend:

```
response = requests.post("http://localhost:8000/generate", json={...})
```

- AI-generated content is sanitized and displayed dynamically:

```
generated_text = sanitize_text(response.json()["document"])
```

### Step 2: HTML Preview Rendering

- The output is previewed in an elegant dark-themed scrollable card with semantic HTML:

```
styled_html = format_html_preview(generated_text)
st.markdown(f"<div style='...'>{styled_html}</div>", unsafe_allow_html=True)
```

### Step 3: Editable Document Preview

- The user can click "Edit Document" and modify the generated content using a text area:

```
edited_text = st.text_area("Edit Document Below:", generated_text, height=300)
```

### Step 4: Multi-Format Download Options

- Users can download the document as:
  - .txt (Plain Text)
  - .docx (Formatted Word Document with logo, headings, and table)
  - .pdf (Branded PDF with logo and footer)

```
st.download_button("📄 Download as .TXT", data=generated_text, file_name=f"{document_type.replace(' ', '_').lower()}.txt", type="text")
st.download_button("📄 Download as .DOCX", data=format_docx(generated_text, document_type), file_name=f"{document_type.replace(' ', '_').lower()}.docx", type="text")
st.download_button("📄 Download as .PDF", data=format_pdf(generated_text, document_type), file_name=f"{document_type.replace(' ', '_').lower()}.pdf", type="text")
```

## Core Functions & Utilities

### **sanitize\_text(text)**

- Removes special characters and typographic quotes to ensure clean formatting.

### **format\_docx(text, doc\_type)**

- Uses python-docx to:
  - Embed logo
  - Add titles, paragraph formatting
  - Auto-generate a Terms table from semicolon-separated input

### **format\_pdf(text, doc\_type)**

- Utilizes FPDF with custom header/footer:
  - Center-aligned logo
  - Bold headings for sections
  - Bullet-style terms

### **format\_html\_preview(text)**

- Converts output to stylized HTML blocks for inline display.